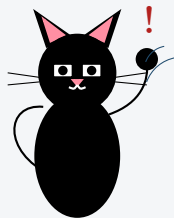


Resumen de Contenidos

IIC2233 — Programación Avanzada

Resumen elaborado por el equipo de ayudantía, en base al material oficial del curso



Felipín otra vez. Esta vez el problema es que Felipín **ignora los errores**: cuando su programa falla, envuelve todo en un enorme `except Exception` sin preguntarse por qué falló. Además, modifica listas mientras las recorre con `for`, hardcodea índices en vez de usar iteradores, y cuando necesita estructuras más complejas, copia y pega nodos a mano sin **encadenarlos** correctamente.

No seas como Felipín. Las herramientas de este resumen — excepciones, iterables/iteradores/generadores y listas ligadas — existen precisamente para que tu código **anticipe errores, recorra datos de forma genérica y ordenada, y modele relaciones entre datos** sin duplicar ni romper nada.

Tabla de Contenidos

1	Introducción	2
2	Excepciones	2
2.1	¿Qué es una excepción?	2
2.2	Tipos de excepciones comunes	3
2.3	Levantando excepciones: <code>raise</code>	4
2.4	Manejo de excepciones: <code>try</code> y <code>except</code>	5
2.4.1	Múltiples excepciones con <code>except</code>	6
2.4.2	Flujos complementarios: <code>else</code> y <code>finally</code>	6
2.5	Jerarquía de excepciones	7
2.6	Excepciones personalizadas	7
3	Iterables, iteradores y generadores	8
3.1	Iterar sobre estructuras de datos	8
3.2	El objeto iterador	9
3.3	Construyendo un iterable y su iterador	10
3.4	Iteradores personalizados	11
3.5	Generadores	11

4	Listas ligadas	12
4.1	El nodo	12
4.2	La lista ligada	13
4.2.1	agregar(valor)	13
4.2.2	obtener(posicion)	14
4.2.3	insertar(valor, posicion)	14
4.3	Iterar sobre listas ligadas	14
4.4	Otras estructuras basadas en nodos	16
5	Ejercicios	16

1 Introducción

Este resumen agrupa tres herramientas fundamentales de Python que, a primera vista, parecen no tener relación entre sí, pero que en la práctica se usan constantemente en conjunto:

■ Los tres bloques de este resumen

- **Excepciones:** cómo Python nos avisa (y cómo nosotros avisamos) cuando algo sale mal durante la ejecución, y cómo **recuperarnos** de esos errores sin que el programa se detenga.
- **Iterables, iteradores y generadores:** cómo recorrer estructuras de datos — propias o de Python — de forma genérica usando `for`, y cómo el mecanismo de iteración **usa excepciones** (`StopIteration`, `IndexError`) para saber cuándo detenerse.
- **Listas ligadas:** una estructura de datos basada en **nodos** encadenados entre sí, que sirve de base para entender estructuras más complejas, y que aprenderemos a recorrer usando exactamente el protocolo de iteración de la sección anterior.

El hilo conductor es la **robustez**: un programa bien escrito anticipa errores (excepciones), recorre sus datos sin asumir una estructura fija (iterables) y organiza información compleja en piezas simples y conectadas (nodos). Vamos a revisar cada uno en profundidad.

2 Excepciones

2.1 ¿Qué es una excepción?

En Ciencia de la Computación, una **excepción** es una condición anómala o inesperada que ocurre durante la ejecución de un programa y que interrumpe su **ejecución esperada**. Cuando esto sucede, el sistema genera un evento llamado **excepción**, que altera el curso habitual del programa.

Entre las situaciones más comunes que generan excepciones se encuentran:

■ Situaciones típicas que generan excepciones

- Operaciones no definidas (ej. división por cero).
- Acceso a regiones de memoria no permitidas (ej. leer más allá de los límites de una lista).
- Acceder a una *key* inexistente en un diccionario.
- Construir un número desde un *string* con formato incorrecto: `int("34C")`.
- Usar una variable no definida, o invocar un método inexistente en un objeto.
- Realizar una operación prohibida para un tipo de dato (ej. modificar una tupla).

Estos casos **podrían** manejarse con estructuras `if/elif/else`, pero eso complica el código: cada nueva condición obliga a modificar múltiples partes del programa, aumentando el riesgo de errores. Por eso, Python permite **generar** (*raise*) excepciones cuando ocurre algo anómalo, y **capturarlas** (*catch*) en bloques diseñados para tratarlas. A este proceso se le llama **manejo de excepciones** (*exception handling*), y permite corregir el problema, notificar al usuario, ignorarlo y continuar, o tomar cualquier acción que permita retomar la ejecución normal.

! ¿Qué pasa si nadie maneja la excepción?

Si una excepción no es manejada, se reporta al sistema operativo y, por lo general, el programa finaliza abruptamente: el intérprete detiene la ejecución y muestra un mensaje indicando el tipo de error ocurrido. En Python, las excepciones son **objetos** que heredan de la clase `Exception`, y se crean en el momento en que se detecta el problema.

2.2 Tipos de excepciones comunes

Todas las excepciones que revisamos a continuación son clases que heredan de `Exception`. Cada una nos entrega información distinta sobre **qué tipo** de problema ocurrió, lo que después nos permitirá manejarlas de forma selectiva.

■ Resumen de excepciones comunes

- **SyntaxError**: una sentencia está mal escrita y viola las reglas sintácticas del lenguaje (ej. olvidar cerrar un paréntesis, un `:` faltante). Se detecta **antes** de ejecutar el programa, al leer todo el código.
- **IndentationError**: hereda de `SyntaxError`. Ocurre cuando una línea no respeta la indentación esperada dentro de un bloque (`if`, `for`, `def`, etc.).
- **NameError**: se intenta usar un nombre (variable, función, clase) que no ha sido definido en el ámbito local o global.
- **ZeroDivisionError**: el denominador de una división es cero. Solo se detecta en **tiempo de ejecución**.
- **IndexError**: se accede a un índice fuera del rango válido de una secuencia (listas, tuplas, etc.).
- **KeyError**: se accede a una *key* inexistente en un diccionario.
- **AttributeError**: se usa un método o atributo que no existe en una clase o tipo de dato.
- **TypeError**: se opera con un argumento de un tipo no adecuado para la operación (ej. sumar un `int` con una `list`).
- **ValueError**: se opera con un argumento del tipo correcto, pero cuyo **valor** no es apropiado (ej. `"abc".split("")`).

```
# IndentationError: falta indentar dentro del bloque if
user = "Alice"
if user != "Queen of Hearts":
    # Un comentario NO cuenta como instruccion,
    # se necesita 'pass' u otra sentencia real.
print("Watch out!")

# ZeroDivisionError: solo se detecta en tiempo de ejecucion
def divide(x, y):
```

```

    return x / y

divide(5, 0) # No se puede anticipar leyendo solo la funcion

# TypeError: duck-typing hace que "+" no siempre este definido
def combine(x, y):
    return x + y

combine(2, [36, 23, 12]) # int + list --> TypeError

```

¿Qué hace este código? El primer bloque falla porque el comentario no cuenta como instrucción real dentro del `if`, dejando el bloque vacío. El segundo falla porque `divide` solo puede detectar el denominador cero **cuando se ejecuta**, no al leer su definición. El tercero muestra *duck-typing*: `combine` usa `+` sin preguntar tipos, así que funciona para dos `int` o dos `list`, pero falla al mezclar un `int` con una `list`.

★ ValueError vs. TypeError

De cierta forma, todo `TypeError` es un `ValueError` (ambos indican que un argumento no es apropiado), pero no todo `ValueError` es un `TypeError`: en `TypeError` el problema es el **tipo** del argumento; en `ValueError` el tipo es correcto pero el **valor** no lo es (ej. `lista.remove(6)` cuando `6` no está en la lista). Se prefiere lanzar el error **más específico** posible, porque entrega más información para manejarlo después.

2.3 Levantando excepciones: raise

Además de las excepciones que Python levanta automáticamente, podemos **generar** una excepción nosotros mismos en el momento que queramos, creando una nueva instancia de la excepción y usando la sentencia `raise`. Cada excepción tiene un tipo definido, y podemos incluir un mensaje explicativo para quien la reciba.

```

def convert_coordinate(coordinate_string):
    # Si el input no es del tipo esperado
    if not isinstance(coordinate_string, str):
        raise TypeError("Coordinate must be a string")

    # Si el input no sigue el formato esperado
    if "," not in coordinate_string:
        raise ValueError("Coordinate must be separated by a comma")

    coord_x, coord_y = coordinate_string.split(",")
    return (int(coord_x), int(coord_y))

```

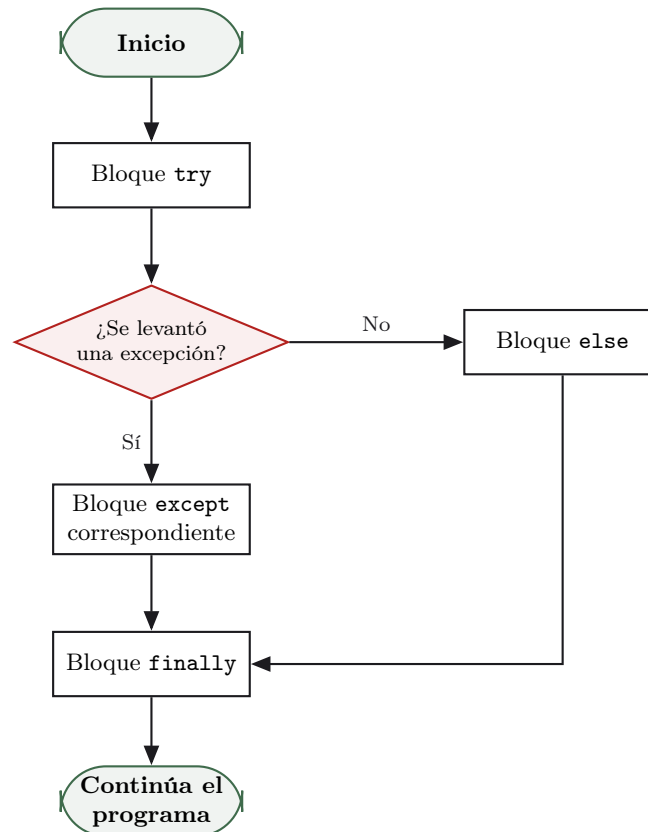
¿Qué hace este código? La función revisa dos condiciones **antes** de intentar procesar el dato: primero que el argumento sea un `str` (si no, `raise TypeError`), y luego que contenga una coma (si no, `raise ValueError`). Solo si ambas validaciones pasan, separa el texto por la coma y castea cada parte a `int`.

! El raise interrumpe TODO el flujo

Una excepción interrumpe el flujo del programa por completo. Aunque ocurra dentro de una función llamada desde otra función (y esta desde otra más), **todas** las llamadas previas se interrumpen y la excepción se propaga hasta el sistema operativo, a menos que sea capturada antes con `try/except`. Por eso, en el ejemplo de `convert_coordinate`, cualquier código **después** del llamado con datos inválidos nunca llega a ejecutarse.

2.4 Manejo de excepciones: try y except

Cada vez que se **levanta** una excepción durante la ejecución, es posible **atraparla** con `try/except`. La sentencia `try` define un bloque de código: si se levanta una excepción dentro de él, esta es **capturada**, y el flujo salta inmediatamente al bloque `except` correspondiente. Una vez que ese bloque termina, el programa continúa en la instrucción **posterior** al `try/except` completo — **nunca** regresa a la línea que gatilló la excepción.



```

r = 4
try:
    # Código que PODRIA arrojar una excepcion
    print(divide(5, r))
    print("I already did the division.")

except (ZeroDivisionError) as error:
    # 'error' es el objeto excepcion, podemos leer su mensaje
    print(f"Error: {error}")
    print("How could you think of dividing by zero?")

print("The program continues after the try/except")
  
```

¿Qué hace este código? `divide(5, r)` se ejecuta dentro del bloque `try`. Como `r` vale 4, no ocurre ninguna excepción: el `except` nunca se ejecuta, y el programa sigue de largo hasta el último `print`. Si `r` hubiese sido 0, el flujo habría saltado inmediatamente al bloque `except` `ZeroDivisionError`, imprimiendo el mensaje de error en vez del resultado.

! SyntaxError e IndentationError NO se pueden atrapar así

Solo se atrapan excepciones que surgen **durante la ejecución** del código. Como `SyntaxError` e `IndentationError` surgen al **leer** el programa, no es posible atraparlas dentro del mismo archivo. Una forma de sí lograrlo es dejar el `import` de un archivo con errores de sintaxis dentro

de un `try/except`: recién durante la ejecución del `import` se intenta leer ese archivo.

2.4.1 Múltiples excepciones con `except`

Podemos definir varios bloques `except`, cada uno manejando un tipo distinto de excepción, o agrupar varios tipos en un mismo bloque entregando una **tupla** de tipos.

```
def divide(num, den):
    if not (isinstance(num, int) and isinstance(den, int)):
        raise TypeError("Type error in numerator or denominator.")
    if num < 0 or den < 0:
        raise ValueError("There is a negative value.")
    return float(num) / float(den)

for arguments in [(45, "5"), (-23, 5), (4, 0), (21, 7)]:
    try:
        print(divide(*arguments))

    # Un solo bloque puede manejar varios tipos, usando una tupla
    except (ZeroDivisionError, TypeError) as error:
        print(f"Error: {error}")
        print("Check the division data.")

    except ValueError as error:
        print(f"Error: {error}")
        print("A ValueError occurred.")
```

¿Qué hace este código? Se prueban cuatro pares de argumentos distintos. `(45, "5")` lanza `TypeError` (el denominador es un `str`), `(-23, 5)` lanza `ValueError` (hay un negativo), `(4, 0)` lanza `ZeroDivisionError`, y `(21, 7)` no lanza nada. El primer `except` agrupa `ZeroDivisionError` y `TypeError` en una sola tupla porque ambos reciben el mismo tratamiento; `ValueError` se maneja aparte, con un mensaje distinto.

2.4.2 Flujos complementarios: `else` y `finally`

El bloque `try/except` puede complementarse opcionalmente con:

■ `else` y `finally`

- **`else`**: se ejecuta solo si no se levantó ninguna excepción dentro del `try`.
- **`finally`**: se ejecuta **siempre**, haya ocurrido o no una excepción. Es ideal para tareas de limpieza, como cerrar un archivo.

```
from os import path

fid = open(path.join("data", "log.txt"), "w")
try:
    for i in range(5, -1, -1):
        fid.write(f"{divide(10, i)}")

except (ZeroDivisionError, TypeError):
    print("Error! Check the input data.")

else:
    # Solo se ejecuta si NO hubo excepciones en el try
    print("The file was created successfully!")
```

```
finally:
    # Se ejecuta SIEMPRE, haya o no excepcion
    print("Always remember to close your files")
    fid.close()
```

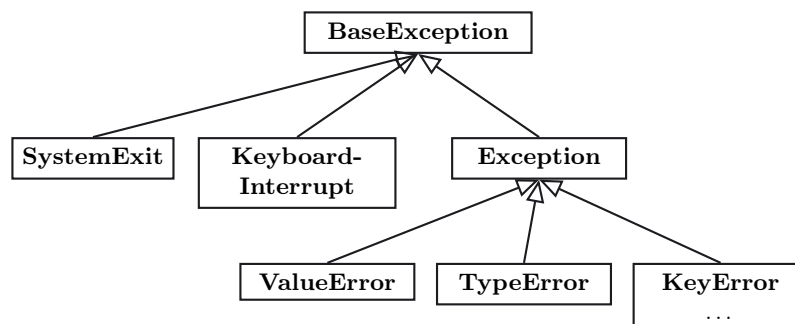
¿Qué hace este código? El `for` intenta escribir seis divisiones en el archivo, y `divide(10, 0)` lanza `ZeroDivisionError` a mitad de camino. Como sí hubo excepción, el bloque `except` se ejecuta y el `else` se salta. El `finally`, en cambio, se ejecuta de todas formas, cerrando el archivo aunque el proceso haya fallado a medio camino.

★ Una alternativa: with

Una forma equivalente y más idiomática de asegurar que un archivo se cierre correctamente es usar un *context manager* con la sentencia `with`: `with open(...) as fid:`. El archivo se cierra automáticamente al salir del bloque, incluso si ocurre una excepción dentro de él, sin necesidad de un `finally` explícito.

2.5 Jerarquía de excepciones

En Python, todas las excepciones heredan de `BaseException`. Desde ella existen tres ramas: `SystemExit`, `KeyboardInterrupt` y `Exception`. Todas las excepciones generadas por errores durante la ejecución de un programa (las que hemos visto en la Sección 2.2) son subclases de `Exception`.



! Capturar Exception a secas es una mala práctica

Como todas las excepciones de ejecución heredan de `Exception`, un bloque `except Exception` captura **cualquier** error, sin importar su naturaleza. Esto puede parecer conveniente, pero es una **mala práctica**: capturar una excepción sin saber qué tipo específico fue puede esconder errores inesperados y causar comportamientos difíciles de depurar. Se recomienda actuar de forma **selectiva**, capturando el tipo más específico posible (`IOError`, `AttributeError`, `ValueError`, etc.).

★ ¿Por qué preferir excepciones sobre if/else?

Podríamos crear un sistema de códigos de error manejado con `return` y muchas combinaciones de `if/elif/else`, pero esto genera casos particulares que ensucian el diseño y son difíciles de mantener. Usar excepciones **obliga** al programador a darles algún tratamiento (manejarlas o levantarlas), mantiene el código más limpio, generaliza el manejo de errores para todo el programa, y permite “notificar” a otras aplicaciones sobre estos problemas de forma estándar.

2.6 Excepciones personalizadas

Podemos crear nuestros propios tipos de excepciones heredando de `Exception`, y modificar su comportamiento sobrescribiendo los métodos que ya tiene implementados (típicamente `__init__`).

```

class TransactionError(Exception):

    def __init__(self, funds, expense):
        super().__init__(f"The money in the wallet is not enough to pay ${expense}")
        self.funds = funds
        self.expense = expense

    def excess(self):
        return self.expense - self.funds

class Wallet:

    def __init__(self, money):
        self.funds = money

    def pay(self, expense):
        if self.funds - expense < 0:
            raise TransactionError(self.funds, expense)
        self.funds -= expense

w = Wallet(1000)
try:
    w.pay(1500)
except TransactionError as err:
    print(f"Error: {err}. There is an overspending of ${err.excess()}")

```

¿Qué hace este código? `TransactionError` sobrescribe `__init__` para armar un mensaje personalizado y guardar `funds`/`expense` como atributos propios; también define `excess()`, un método propio de esta excepción. Al intentar pagar 1500 con solo 1000 de fondos, `Wallet.pay` levanta la excepción; el bloque `except` la captura y usa `err.excess()` para reportar exactamente cuánto faltó.

★ ¿Por qué crear excepciones propias?

Una excepción personalizada no solo lleva un mensaje: puede tener **atributos y métodos propios** (como `excess()` en el ejemplo), lo que permite recuperar información específica del error en el bloque `except`. También podemos usar **atributos de clase** para llevar un registro compartido entre todas las instancias de la excepción — por ejemplo, para acumular todos los duplicados detectados al inscribir estudiantes en un curso.

3 Iterables, iteradores y generadores

3.1 Iterar sobre estructuras de datos

Estructuras como listas, tuplas, *sets* y diccionarios comparten la noción de que pueden ser **iteradas** (recorridas). Para poder crear nuestras propias estructuras que también se puedan recorrer con `for`, necesitamos entender dos conceptos clave: **iterable** e **iterador**.

■ Definiciones clave

- Un **iterable** es cualquier objeto **sobre el cual se puede iterar**: se pueden ir obteniendo elementos paulatinamente. Es lo que puede aparecer a la derecha del `in` en un `for i in iterable:`.

- Un **iterador** es un objeto **que itera sobre un iterable**: es el objeto que retorna el método `__iter__()`, y sabe avanzar de un elemento al siguiente mediante `__next__()`.

Un objeto se considera **iterable** de dos formas posibles en Python:

■ Dos caminos para ser iterable

1. **Implementando `__iter__()`**: se puede iterar todas las veces que se quiera (como las listas). No es necesario que el objeto se pueda indexar con `[]` — por ejemplo, los *sets* no se indexan, pero sí se pueden iterar.
2. **Implementando `__getitem__()`**: debe representar la posición de lo que se desea acceder. Cuando se pide una posición inválida, **debe** levantar un `IndexError`; con eso finaliza la iteración.

```
# Ejemplo 1: implementa __iter__, no __getitem__
conjunto = {1, 3, 4, 6}
for i in conjunto:
    print(i, end=" ")

# Ejemplo 2: implementa __getitem__, no __iter__
class DuplicarCaracteresPalabra:
    def __init__(self, palabra_original: str) -> None:
        self._palabra = palabra_original

    def __getitem__(self, index: int) -> str:
        # La palabra original levanta IndexError
        # cuando se accede a un indice invalido
        return self._palabra[index] * 2

iterable = DuplicarCaracteresPalabra('.py')
for elemento in iterable:
    print('->', elemento)
# El for termina cuando __getitem__ levanta IndexError
```

¿Qué hace este código? En el primer ejemplo, el `for` recorre el *set* usando su `__iter__` interno, sin que nosotros hagamos nada especial. En el segundo, `DuplicarCaracteresPalabra` no implementa `__iter__`: el `for` entonces prueba `__getitem__(0)`, `__getitem__(1)`, etc., y se detiene automáticamente en el índice donde `self._palabra[index]` lanza `IndexError`.

3.2 El objeto iterador

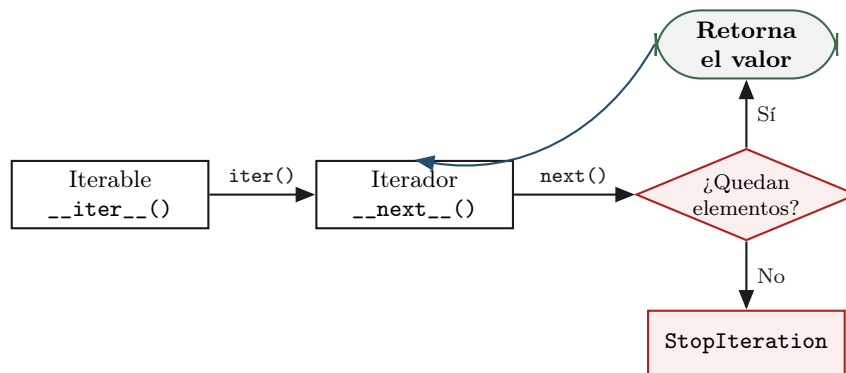
Un **iterador** es el objeto retornado por `__iter__()`. Implementa `__next__()`, que retorna uno a uno los elementos de la estructura cada vez que se invoca. Cuando no quedan elementos, el iterador **debe** levantar un `StopIteration`.

```
conjunto = {1, 4, 6}
iterador = iter(conjunto) # equivalente a conjunto.__iter__()
print(type(iterador))

print(next(iterador)) # equivalente a iterador.__next__()
print(iterador.__next__())
print(next(iterador))
print(next(iterador)) # StopIteration: ya no quedan elementos
```

¿Qué hace este código? `iter(conjunto)` pide al *set* su iterador (llama a `__iter__()` internamente). Cada llamado a `next(iterador)` avanza una posición y retorna un valor nuevo. Como

el *set* solo tiene tres elementos, el cuarto llamado a `next` ya no tiene nada que retornar y levanta `StopIteration`.



★ Resumen en dos frases

Un **iterable** es un objeto que contiene una colección de elementos y puede recorrerse uno a uno. Un **iterador** es un objeto que **avanza** sobre un iterable, de un elemento al siguiente, hasta agotarlos. Un **iterable** se puede recorrer múltiples veces (su `__iter__` siempre retorna un iterador **nuevo**); un **iterador** solo puede recorrerse **una única vez**.

3.3 Construyendo un iterable y su iterador

Para crear una estructura iterable propia, se implementan típicamente **dos clases**: una clase **iterable** (con `__iter__`) y una clase **iteradora** (con `__iter__` que retorna `self`, y `__next__`).

```

from typing import Self

class IterableListaNumeros:
    def __init__(self, objeto: list) -> None:
        self.objeto = objeto

    def __iter__(self) -> "IteradorListaNumeros":
        # Cada llamado retorna un iterador NUEVO
        return IteradorListaNumeros(self.objeto)

class IteradorListaNumeros:
    def __init__(self, iterable: list) -> None:
        # Copiamos para no afectar los valores originales
        self.iterable = iterable.copy()

    def __iter__(self) -> Self:
        return self

    def __next__(self) -> int:
        if not self.iterable:
            raise StopIteration("Llegamos al final")
        else:
            valor = self.iterable.pop(0)
            return valor
  
```

¿Qué hace este código? `IterableListaNumeros.__iter__` no recorre nada por sí mismo: solo **fabrica y retorna** un `IteradorListaNumeros` nuevo, pasándole la lista original. El iterador guarda su **propia copia** de la lista y, en cada `__next__`, extrae y retorna el primer valor con `pop(0)`; cuando la copia queda vacía, levanta `StopIteration` para señalar el final.

! Iterable una vez, iterador solo una vez

Si iteramos sobre un `IterableListaNumeros`, podemos hacerlo **tantas veces como queramos**, porque cada `for` pide un iterador **nuevo** mediante `__iter__`. En cambio, si iteramos directamente sobre una instancia de `IteradorListaNumeros`, esta solo funcionará **una vez**: una vez agotada, no se puede reiniciar. Además, cada iterador guarda su propio progreso de forma **independiente** — dos iteradores del mismo iterable pueden estar en posiciones distintas al mismo tiempo, y es posible interrumpir un recorrido (`break`) y continuarlo después usando el mismo objeto iterador.

3.4 Iteradores personalizados

Un iterador no solo permite recorrer los elementos de un iterable: también permite **definir el orden** en que se recorren. Aprovechando la herencia, podemos crear variantes que ordenen o mezclen los datos antes de empezar a iterar.

```
from random import shuffle
from typing import Self

class IteradorListaNumerosOrdenada(IteradorListaNumerosOriginal):
    def __iter__(self) -> Self:
        self.iterable.sort() # Ordena antes de recorrer
        return self

class IteradorListaNumerosAleatoria(IteradorListaNumerosOriginal):
    def __iter__(self) -> Self:
        shuffle(self.iterable) # Mezcla antes de recorrer
        return self
```

¿Qué hace este código? Ambas clases **heredan** de `IteradorListaNumerosOriginal` y reutilizan su `__next__` tal cual; solo sobrescriben `__iter__` para modificar `self.iterable` (ordenándola o mezclándola) **justo antes** de empezar a iterar. El resto de la lógica de recorrido no cambia en absoluto.

Este patrón no se limita a listas de números: también aplica a tuplas, *sets*, *strings*, u objetos de nuestras propias clases, ordenando o filtrando según cualquier criterio que definamos.

3.5 Generadores

Los **generadores** son un caso especial de los iteradores: permiten iterar sobre secuencias de datos **sin almacenarlos** en una estructura, evitando un uso innecesario de memoria. Una vez que terminamos de iterar sobre un generador, este **desaparece**.

La sintaxis para crear un generador es muy parecida a la comprensión de listas, pero usando paréntesis normales `()` en vez de corchetes `[]`:

```
generador_pares = (2 * i for i in range(10))
print(type(generador_pares)) # <class 'generator'>

for i in generador_pares:
    print(i, end=" ")
# Los generadores implementan __iter__ retornando self,
# por eso funcionan directamente en un for.

# Al agotarse, ya no entrega mas valores:
for i in generador_pares:
    print(i, end=" ") # No imprime nada, ya se agoto
```

¿Qué hace este código? `(2 * i for i in range(10))` crea un objeto **generator**, sin calcular nada todavía. El primer **for** sí recorre y muestra los diez pares porque el generador implementa `__iter__` retornando `self`. El segundo **for** no imprime nada, porque el generador — igual que cualquier iterador — ya se agotó tras el primer recorrido.

! Filtrar vs. transformar dentro de un generador

Dentro de un generador podemos usar condiciones de **dos formas** distintas:

- Un **if al final filtra** elementos: solo se generan los que cumplen la condición.
- Un **if-else dentro** de la expresión **transforma** valores: se generan **todos**, pero algunos se modifican.

```
(i for i in range(10) if i < 3) # FILTRA: 0 1 2
(i if i < 3 else 3 for i in range(10)) # TRANSFORMA: 0 1 2 3 3 3 3 3 3 3
```

★ La gran ventaja: memoria

Usando `sys.getsizeof`, se observa que un generador ocupa una fracción mínima de la memoria que ocuparía una lista con los mismos resultados — y esta diferencia crece mientras más elementos se generan. Esto es porque el generador **produce** cada valor cuando se le solicita, en vez de mantener todos los valores en memoria simultáneamente. Es especialmente útil para leer archivos grandes línea por línea, en vez de cargarlos completos con `archivo.readlines()`.

4 Listas ligadas

Python ofrece estructuras de datos genéricas (listas, tuplas, *sets*, diccionarios), pero en ocasiones estas no son suficientemente expresivas para representar relaciones complejas entre datos. Las **listas ligadas** son la base para representar agrupaciones de datos y sus relaciones usando OOP.

4.1 El nodo

El **nodo** es la unidad indivisible que contiene **datos**, identificable mediante un atributo llave. Cada nodo mantiene cero o más referencias a otros nodos **vecinos**. Un conjunto de nodos relacionados entre sí permite definir estructuras de datos más complejas y expresivas.

```
class Nodo:
    """Representa un nodo de una lista ligada"""

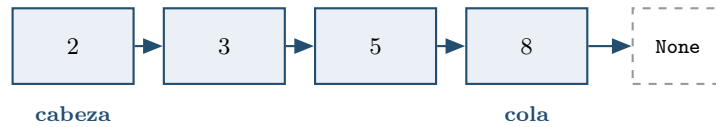
    def __init__(self, valor=None) -> None:
        self.valor = valor
        self.siguiente = None # Referencia al siguiente nodo, inicia vacia

    def __repr__(self) -> str:
        return f"Nodo[{self.valor}]"
```

¿Qué hace este código? La clase `Nodo` solo guarda dos cosas: el **valor** que le corresponde, y una referencia **siguiente** que **empieza en None** porque, al crear un nodo, todavía no sabemos a qué otro nodo se conectará. El método `__repr__` solo existe para que se vea legible al imprimirlo (ej. `Nodo[8]`).

4.2 La lista ligada

Una **lista ligada** almacena nodos en orden secuencial, donde cada nodo posee una referencia a un único nodo **sucesor** (*next node*). El primer nodo se llama **cabeza** (*head*); el último, que no tiene sucesor (**siguiente** es `None`), se llama **nodo cola** (*tail*).



! “Cola” no es lo mismo que *queue*

El término **cola** aquí se refiere al **último elemento** de la lista (*tail*), y no a la estructura secuencial FIFO que en inglés se llama *queue*. Para evitar confusiones, en este resumen hablamos de **nodo cola**.

La clase `ListaLigada` referencia **directa y únicamente** a los nodos **cabeza** y **cola** de la secuencia. Para acceder a cualquier otro nodo, es necesario recorrer las referencias desde la cabeza. Definiremos tres métodos principales:

■ Métodos de `ListaLigada`

- `agregar(valor)`: agrega un nodo nuevo al **final** de la lista (similar a `list.append`).
- `obtener(posicion)`: recupera el valor del nodo en la posición dada, recorriendo la lista desde la cabeza.
- `insertar(valor, posicion)`: agrega un nodo nuevo en una posición arbitraria.

4.2.1 `agregar(valor)`

■ Pasos de `agregar`

1. Crear un nodo nuevo con el valor dado.
2. Al nodo **cola** actual, añadirle como sucesor el nodo recién creado.
3. Actualizar el atributo `cola` de la lista ligada, para que apunte al nodo nuevo.

```

def agregar(self, valor):
    nuevo = Nodo(valor)

    if self.cabeza is None:
        # Lista vacía: el nuevo nodo es cabeza y cola a la vez
        self.cabeza = nuevo
        self.col = self.cabeza
    else:
        # Enlazamos el nodo nuevo como sucesor de la cola actual
        self.col.siguiente = nuevo
        self.col = self.col.siguiente
    self.largo += 1
  
```

¿Qué hace este código? Si la lista está vacía (`self.cabeza is None`), el nodo nuevo pasa a ser **ambos**, cabeza y cola. Si ya había elementos, se conecta el nodo nuevo como **siguiente** de la cola actual, y **luego** se actualiza `self.col` para que apunte a él. El orden importa: si actualizáramos `self.col` antes de enlazar, perderíamos la referencia al nodo cola anterior.

4.2.2 obtener(posicion)

A diferencia de `list`, no hay acceso indexado directo: es necesario **recorrer** cada nodo, comenzando desde la cabeza, siguiendo la referencia `siguiente` hasta llegar a la posición deseada.

```
def obtener(self, posicion):
    if posicion < 0:
        return None

    nodo_actual = self.cabeza
    for _ in range(posicion):
        if nodo_actual is not None:
            nodo_actual = nodo_actual.siguiente

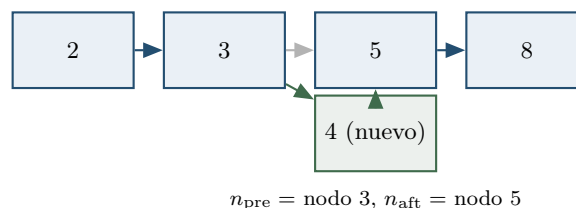
    if nodo_actual is None: # Posicion fuera de rango
        return None
    return nodo_actual.valor
```

¿Qué hace este código? Partiendo desde `self.cabeza`, el `for` avanza `posicion` veces siguiendo `nodo_actual.siguiente`, cuidando de no seguir avanzando si ya se llegó a `None` (fin de la lista). Si tras el recorrido `nodo_actual` terminó siendo `None`, significa que la posición pedida no existía, y se retorna `None` en vez de lanzar un error.

4.2.3 insertar(valor, posicion)

■ Pasos de insertar

1. Crear un nodo nuevo con el valor dado.
2. Buscar el nodo que quedará **después** del nuevo (n_{aft}): el que actualmente está en `posicion`.
3. Buscar el nodo que quedará **antes** del nuevo (n_{pre}): el nodo en `posicion - 1`.
4. Hacer que el `siguiente` del nodo nuevo sea n_{aft} .
5. Hacer que el `siguiente` de n_{pre} sea el nodo nuevo.



★ Insertar en una lista ligada es más barato que en una lista de Python

Insertar un nodo en una posición arbitraria de una lista ligada solo requiere **reconectar dos referencias**, independiente del tamaño de la lista (aunque **encontrar** esa posición sí requiere recorrerla desde la cabeza). En cambio, insertar en medio de una `list` de Python requiere **desplazar** todos los elementos posteriores una posición.

4.3 Iterar sobre listas ligadas

Como vimos, para recorrer manualmente una lista ligada hay que acceder nodo por nodo desde la cabeza:

```
nodo_actual = ll.cabeza
while nodo_actual is not None:
```

```
print(nodo_actual, end=" ")
nodo_actual = nodo_actual.siguiente
```

¿Qué hace este código? `nodo_actual` arranca en la cabeza, y en cada vuelta del `while` se imprime y se avanza con `nodo_actual = nodo_actual.siguiente`. El ciclo termina solo cuando `nodo_actual` llega a `None`, es decir, cuando pasamos el nodo cola.

Pero aplicando lo aprendido en la Sección 3, podemos adaptar `ListaLigada` para que se pueda recorrer con `for`, implementando una clase `iterador` y una clase `iterable`:

```
from __future__ import annotations
from typing import Any

class IteradorListaLigada:
    def __init__(self, cabeza: "Nodo") -> None:
        self.nodo_actual = cabeza

    def __iter__(self) -> IteradorListaLigada:
        return self

    def __next__(self) -> Any:
        if self.nodo_actual is None:
            raise StopIteration("Llegamos al final")
        else:
            nodo = self.nodo_actual
            self.nodo_actual = self.nodo_actual.siguiente
            return nodo

class IterableListaLigada(ListaLigada):
    def __iter__(self) -> IteradorListaLigada:
        # Cada for pide un iterador nuevo, empezando desde la cabeza
        return IteradorListaLigada(self.cabeza)
```

¿Qué hace este código? `IteradorListaLigada` no guarda una copia de nada: simplemente recuerda `nodo_actual`, y en cada `__next__` retorna ese nodo y avanza a su `siguiente`, hasta toparse con `None` (ahí levanta `StopIteration`). `IterableListaLigada` hereda todo el comportamiento de `ListaLigada` y solo agrega `__iter__`, que crea un iterador nuevo apuntando a la cabeza cada vez que se pide.

```
lista_iterable = IterableListaLigada()
for valor in [2, 3, 5, 8, 13]:
    lista_iterable.agregar(valor)

for nodo in lista_iterable:
    print(nodo, end=" ")
# Nodo[2] Nodo[3] Nodo[5] Nodo[8] Nodo[13]
```

¿Qué hace este código? Se agregan cinco valores con `agregar` (heredado sin cambios) y luego se recorre la lista con un `for` normal, gracias a que `IterableListaLigada` ahora sigue el protocolo de iteración. Internamente, Python llama a `__iter__` una vez al empezar el `for`, y a `__next__` una vez por cada vuelta.

★ Mismo protocolo, dos estructuras distintas

Nótese que `IteradorListaLigada` sigue exactamente el mismo protocolo que vimos para `IteradorListaNumeros` en la Sección 3: implementa `__iter__` (retornando `self`) y `__next__` (levantando `StopIteration` al terminar). El mecanismo de iteración de Python es **genérico**: una vez que entendemos el protocolo, podemos aplicarlo a cualquier estructura de datos, sea

una lista de números o una lista ligada de nodos.

4.4 Otras estructuras basadas en nodos

Las listas ligadas son solo el punto de partida: existen múltiples estructuras de datos basadas en el mismo principio de nodos que se referencian entre sí.

■ Otras estructuras nodales

- **Árboles:** árboles binarios, árboles de búsqueda binaria, *tries*, *heaps* — cada nodo referencia a más de un sucesor.
- **Grafos:** direccionales o bidireccionales — cada nodo puede referenciar a múltiples vecinos sin una jerarquía fija.

! Fuera del alcance de este curso

Profundizar en árboles y grafos queda fuera del alcance de IIC2233. Si te interesa seguir aprendiendo sobre estructuras de datos basadas en nodos, el curso **IIC2133 — Estructuras de Datos y Algoritmos** es el siguiente paso natural.

5 Ejercicios

UNO Solitario

Dificultad: 🐱🐱🐱🐱🐱

■ Enunciado I2-2026-1 ■ 20% nota final

El “UNO Solitario” es un juego de cartas de una persona donde el jugador debe descartar todas las cartas de su mano en un pozo. Para descartar las cartas se debe emparejar — ya sea por número o color — la carta que está más arriba del pozo con la carta descartada. El jugador deberá realizar esta acción hasta que ya no le queden cartas en la mano, o que las cartas que queden no puedan ser jugadas.

Considerando que las cartas del mazo solo contienen cartas numéricas, deberás utilizar correctamente el patrón **Iterable Iterator** para implementar la forma en que se juega las cartas una mano de “UNO Solitario”. Para lograr lo anterior, se representará la **Mano de Cartas** como una **Lista Ligada de Cartas**. Esto se ve reflejado en el siguiente código base:

```
from copy import deepcopy

class Carta:
    def __init__(self, color: str, numero: int) -> None:
        self.color = color
        self.numero = numero
        self.siguiente = None

class ManoCartas:
    def __init__(self) -> None:
        self.cabeza = None
        self cola = None
```



```

def agregar_carta(self, carta: Carta) -> None:
    if self.cabeza is None:
        self.cabeza = carta
        self cola = carta
        return
    self.colasiguiente = carta
    self.colasiguiente = carta

def __iter__(self) -> IteradorManoCartas:
    return IteradorManoCartas(deepcopy(self.cabeza))

class IteradorManoCartas:
    def __init__(self, cabeza: Carta) -> None:
        self.mano = cabeza

if __name__ == '__main__':
    # Se define previamente 'mano_cartas' como una instancia de ManoCartas y
    # se le agregan multiples instancias de Carta
    for carta in mano_cartas:
        print('Carta jugada:', carta.numero, carta.color)

```

Utilizando únicamente la Lista Ligada como estructura de datos, deberás completar el código anterior para permitir que se pueda iterar una instancia de `ManoCartas` asegurando que se cumpla todo lo indicado anteriormente.

Puedes utilizar pseudo-código. Además, puedes agregar nuevos atributos, crear nuevos métodos, clases y funciones siempre y cuando esto no modifique el funcionamiento de lo que ya se encuentra implementado.

✓ Solución

La idea central es que `IteradorManoCartas` **no** recorre la lista ligada de forma pasiva: en cada llamado a `__next__`, debe **buscar** — entre las cartas restantes de la mano — la primera que calce en color o número con la carta que está arriba del pozo, sacarla de la lista ligada, actualizar el pozo con ella, y retornarla. Cuando ninguna carta calza, o cuando ya no quedan cartas en la mano, se levanta `StopIteration` para terminar la iteración, exactamente como pide el protocolo **Iterable Iterator** de la Sección 3.

```

from copy import deepcopy

class Carta:
    def __init__(self, color: str, numero: int) -> None:
        self.color = color
        self.numero = numero
        self.siguiente = None

    def __eq__(self, value: "Carta") -> bool:
        # Dos cartas 'calzan' si comparten color o numero
        return self.color == value.color or self.numero == value.numero

```

```

class ManoCartas:
    def __init__(self) -> None:
        self.cabeza = None
        self cola = None

    def agregar_carta(self, carta: Carta) -> None:
        if self.cabeza is None:
            self.cabeza = carta
            self.col a = carta
            return
        self.col a.sigui ente = carta
        self.col a = carta

    def __iter__(self) -> "IteradorManoCartas":
        # Se itera sobre una COPIA de la mano, para no destruir el original
        return IteradorManoCartas(deepcopy(self.cabeza))

class IteradorManoCartas:
    def __init__(self, cabeza: Carta) -> None:
        self.mano = cabeza # Cabeza de la mano restante por jugar
        self.pozo = cabeza # Carta que esta arriba del pozo

    def __next__(self) -> Carta:
        carta_anterior = None
        carta_actual = self.mano

        # Si no quedan cartas en la mano, se termina la iteracion
        if carta_actual is None:
            raise StopIteration("No quedan cartas en la mano")

        # Recorremos la mano buscando una carta jugable
        while carta_actual is not None:
            # Comprobamos si la carta puede ser jugada (mismo color o numero)
            if carta_actual == self.pozo:
                # Si la carta jugable es la cabeza, se cambia la referencia
                if carta_actual is self.mano:
                    self.mano = carta_actual.sigui ente
                else:
                    # Si no es la cabeza, se saca de la Lista Ligada
                    # reconectando su nodo anterior con su siguiente
                    carta_anterior.sigui ente = carta_actual.sigui ente

                # Actualizamos la carta que queda arriba del pozo
                self.pozo = carta_actual
                carta_actual.sigui ente = None
                return carta_actual

            carta_anterior = carta_actual
            carta_actual = carta_actual.sigui ente

        # Se recorrio toda la mano y ninguna carta calzo con el pozo
        raise StopIteration("No se puede jugar ninguna carta")

```

¿Qué hace este código? `Carta` incorpora `__eq__` para definir qué significa que “calcen” dos cartas (mismo color o mismo número), lo que permite comparar directamente `carta_actual == self.pozo`. `ManoCartas.__iter__` entrega, como exige el patrón **Iterable Iterator**, un

`IteradorManoCartas` **nuevo** cada vez, construido sobre una **copia** de la cabeza — así el `for` del enunciado nunca modifica la mano original. El iterador guarda dos referencias: `self.mano` (lo que falta por revisar) y `self.pozo` (la última carta jugada). En cada `__next__`, recorre la mano restante nodo a nodo hasta encontrar una carta que calce con el pozo; al encontrarla, la desconecta de la lista ligada (cuidando el caso especial de que sea la cabeza) y la retorna como la carta jugada. Si recorre toda la mano sin encontrar ninguna coincidencia, o si la mano ya está vacía, levanta `StopIteration`, deteniendo el `for` tal como exige el protocolo de iteración.

★ Por qué se usa `deepcopy`

`deepcopy(self.cabeza)` es clave para respetar que `ManoCartas` sea un **iterable** y no un **iterador**: como cada `__iter__` debe poder llamarse múltiples veces, y el algoritmo de `__next__` **destruye** nodos de la lista que recorre (los va sacando a medida que los juega), es necesario iterar siempre sobre una copia — de lo contrario, la primera vez que se recorra la mano completa esta quedaría vacía para siempre.

Fin del resumen.